arXiv:2406.16386v3 [cs.SE] 25 Apr 2025

Divide-and-Conquer: Generating UI Code from Screenshots

YUXUAN WAN, The Chinese University of Hong Kong, China
CHAOZHENG WANG, The Chinese University of Hong Kong, China
YI DONG, The Chinese University of Hong Kong, China
WENXUAN WANG, The Chinese University of Hong Kong, China
SHUQING LI, The Chinese University of Hong Kong, China
YINTONG HUO*, Singapore Management University, Singapore
MICHAEL LYU, The Chinese University of Hong Kong, China

Websites are critical in today's digital world, with over 1.11 billion currently active and approximately 252,000 new sites launched daily. Converting website layout design into functional UI code is a time-consuming yet indispensable step of website development. Manual methods of converting visual designs into functional code present significant challenges, especially for non-experts. To explore automatic design-to-code solutions, we first conduct a motivating study on GPT-4o and identify three types of issues in generating UI code: element omission, element distortion, and element misarrangement. We further reveal that a focus on smaller visual segments can help multimodal large language models (MLLMs) mitigate these failures in the generation process.

In this paper, we propose DCGen, a divide-and-conquer-based approach to automate the translation of webpage design to UI code. DCGen starts by dividing screenshots into manageable segments, generating code for each segment, and then reassembling them into complete UI code for the entire screenshot. We conduct extensive testing with a dataset comprised of real-world websites and various MLLMs and demonstrate that DCGen achieves up to a 15% improvement in visual similarity and 8% in code similarity for large input images. Human evaluations show that DCGen can help developers implement webpages significantly faster and more similar to the UI designs. To the best of our knowledge, DCGen is the first segment-aware MLLM-based approach for generating UI code directly from screenshots.

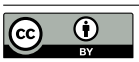
CCS Concepts: • **Software and its engineering** → **Automatic programming**; • **Computing methodologies** → *Artificial intelligence*.

Additional Key Words and Phrases: Multi-modal Large Language Model, Code Generation, User Interface, Web Development

ACM Reference Format:
Yuxuan Wan, Chaozheng Wang, Yi Dong, Wenxuan Wang, Shuqing Li, Yintong Huo, and Michael Lyu. 2025. Divide-and-Conquer: Generating UI Code from Screenshots. *Proc. ACM Softw. Eng.* 2, FSE, Article FSE094 (July 2025), 24 pages. https://doi.org/10.1145/3729364

*Yintong Huo is the corresponding author.

Authors' Contact Information: Yuxuan Wan, The Chinese University of Hong Kong, Hong Kong, China, yxwan9@cse.cuhk.edu.hk; Chaozheng Wang, The Chinese University of Hong Kong, Hong Kong, China, adf111178@gmail.com; Yi Dong, The Chinese University of Hong Kong, Hong Kong, China, 1155173962@link.cuhk.edu.hk; Wenxuan Wang, The Chinese University of Hong Kong, Hong Kong, China, wxwang@cse.cuhk.edu.hk; Shuqing Li, The Chinese University of Hong Kong, Hong Kong, China, sqli21@cse.cuhk.edu.hk; Yintong Huo, Singapore Management University, Singapore, Singapore, ythuo@smu.edu.sg; Michael Lyu, The Chinese University of Hong Kong, Hong Kong, China, lyu@cse.cuhk.edu.hk.



This work is licensed under a Creative Commons Attribution 4.0 International License.
© 2025 Copyright held by the owner/author(s).
ACM 2994-970X/2025/7-ARTFSE094
https://doi.org/10.1145/3729364

1 Introduction

Websites are important in today's digital landscape, serving as essential platforms for diverse applications in our daily lives. Statistics show that there are over 1.1 billion online websites, with approximately 252,000 new websites being created every day [1, 3].

Developing a website's graphical user interface (GUI) involves two sequential stages: UI design and UI implementation. During the design stage, graphic artists create and customize the website layout, which is then provided to developers as pixel-based concept drawings [13, 40]. Subsequently, the implementation stage involves translating these visual designs into functional UI code using GUI frameworks. Screenshots of the GUI can effectively approximate the original UI designs, as they capture the intended pixel-level layout and visual structure of the design before being converted into operational code.

Manually implementing visual designs of websites into functional code is complicated and time-consuming, as it requires domain knowledge of mapping GUI framework elements to spatial layouts. For laypersons, this knowledge barrier has hindered them from efficiently building their own web applications, even if they have concrete ideas for the design [62]. For developers, acquiring such knowledge is challenging, time-consuming, and error-prone due to the intricate nature of GUI frameworks, marked by the excessive amount of GUI components, the complexity of spatial layouts, and the diversity of GUI frameworks [13, 41, 50, 52]. Although modern GUI builders provide interactive features (e.g., drag & drop, what-you-see-is-what-you-get) for developing a GUI, these tools often limit customization [52], causing discrepancies between the intended design and actual implementation, and are known to introduce bugs and presentation failures even for simple tasks [74].

The challenges in GUI production necessitate an automated approach to translating UI designs (e.g., images, Photoshop documents) into GUI code, referred to as *Design-to-Code*. The automated design-to-code techniques offer benefits to individuals and companies. First, it has the potential to democratize front-end web application development [52], allowing non-experts to easily build web applications. Second, it allows smaller companies to focus more on visual features, rather than on translating designs into functional code [49].

Despite its promising benefits, practical solutions for generating code implementation from website design have been underexplored. A line of research made efforts to transform GUI design into code for Android mobile apps but these approaches are not generalizable to the website design containing complicated interfaces [5, 13, 49, 52]. To enable finger-friendly touch targets, mobile apps utilize minimalist designs to avoid clutter on small screens, resulting in fewer visual elements and simpler structures. Additionally, mobile development relies on platform-specific tools [37] (e.g., iOS uses Xcode, Android uses Android Studio), which do not apply to website development that typically uses HTML, CSS, and JavaScript [2, 4, 61]. Another line of research generates website UI code from screenshots using deep learning-based techniques (e.g., CNN, LSTM), but they rely on synthetic data. For example, Beltramelli et al. [11] integrates CNN and RNN models to generate code from screenshots. However, this method is limited to a narrow range of simple user interface designs, accommodating only five types of web elements. Despite training on pairs of screenshots and code, DL-based approaches are still restricted by their training data, which may contain limited web elements and programming knowledge. In a nutshell, existing studies are unsuitable for building real-world software websites.

Multimodal Large Language Models (MLLMs) have seen significant advancements recently [22, 32, 54], offering an alternative solution for design-to-code. By integrating image processing capabilities into large language models (LLMs), MLLMs have demonstrated superior ability than CNNs for understanding images and answering visual questions [71]. Furthermore, research has shown

that LLMs have remarkable performance on various code intelligence tasks [36], including code generation [25, 27, 31, 38, 43, 73], code completion [16, 23, 24, 44, 45, 53], and code summarization [9, 14, 30, 34, 46, 47]. The strong image understanding and code generation capabilities of MLLMs showcase their potential to transform website design images into functional web code.

Unfortunately, notwithstanding MLLMs' impressive performance in image-to-text tasks, directly applying them for UI code generation results in unsatisfactory performance because of the complicated nature of GUI. Unlike generating general descriptions, translating images into UI code poses unique challenges for MLLMs. Firstly, it must accurately detect and classify diverse elements and nested structures within a single webpage, such as buttons, text fields, and images. Secondly, it demands the precise replication of intricate layouts and styles. Modern GUIs cannot be constructed merely through the hardcoded positioning of texts, images, and controls; rather, the model must possess a thorough understanding of a GUI framework's components, including the supported visual effects and compositions, to reproduce elements with the details (e.g., colors, fonts, margins) and positioning [13]. Thirdly, unlike static descriptions, the generated code must be executable, adhering to the syntax and semantic requirements of front-end frameworks.

To understand MLLMs's capability in transforming UI images to UI code, we conduct a motivating study to identify three main issues in this process: (1) element omission, where certain visual components are missing in the generated code, (2) element distortion, where elements are inaccurately reproduced in terms of shape, size, or color, and (3) element misarrangement, meaning the elements are not positioned or ordered correctly relative to their design layout. In contrast, when we cropped the inaccurately generated images into smaller pieces and asked MLLMs to re-generate UI code for each segment, we observed enhanced performance on these smaller, cropped UI images. *In short, the smaller and more focused the image, the better the resulting code quality.*

Building on this insight, we propose a novel **D**ivide-and-**C**onquer-based method to **Gen**erate website code from UI designs, namely **DCGen**. This end-to-end approach follows the principles of a typical divide-and-conquer algorithm: breaking down a complex problem into smaller parts, solving each part individually, and then combining the solutions to address the original problem. Specifically, DCGen consists of two phases: *division* and *assembly*. The first phase involves slicing the design images into smaller, more manageable, yet semantically meaningful pieces, and generating the HTML and CSS code for each distinctive segment. The division aligns with the real-world front-end development practices [19] and is achieved via a novel image segmentation algorithm. DCGen then generates code for each segment, followed by recursively reassembling the code to reconstruct the entire website. This reassembly process is the reverse of the division procedure, where code from smaller, child segments is progressively integrated to build up their parent segments. This assembly continues until the full website structure is restored.

Our study curated a dataset comprising 348 top-ranked real-world websites. We evaluate the effectiveness of our methodology across several cutting-edge MLLMs, using 111 webpages sampled from the dataset. DCGen achieved up to 8% improvement in visual similarity and 15% improvement in code similarity for large input images compared to other design-to-code methods. We further demonstrate that DCGen is robust against complex webpages. Moreover, DCGen is shown to generalize well across different MLLMs, demonstrating the effectiveness of the divide-and-conquer methodology. Human evaluations show that DCGen can produce more visually similar target webpages than competing methods and can help developers implement webpages faster.

In summary, our paper makes the following contributions:

- We initiated a motivating study to uncover types of errors in the MLLM-powered design-to-code process, revealing the importance of visual segments in generation quality.

- We propose DCGen with a divide-and-conquer approach. It starts with generating code for individual image segments, and then merges these solutions to produce the complete UI code.
- The experiments on real-world webpages demonstrate the superiority of DCGen over other methods in both visual similarity and code similarity. Human evaluations also confirm that DCGen can help developers implement webpages faster and more similar to the UI designs.
- We implement DCGen into a user-friendly demo tool and release all datasets and code implementation for future research in this field.

2 Background

Concepts in Front-End Development. Front-end development concentrates on creating the user interface and enhancing the user experience of websites. It involves Hypertext Markup Language (HTML) for content structuring, Cascading Style Sheets (CSS) for styling, and optionally JavaScript for interactivity and dynamic webpage features. HTML is a markup language used in front-end web development to structure webpages. Tags in HTML define the document's structure, like `<h1>` to `<h6>` for different heading levels, and `<p>` for paragraphs. Each HTML element includes an opening tag, content, and a closing tag, forming the basic block of a webpage. CSS is used in front-end web development to style webpages and enhance the appearance of HTML elements. It allows developers to define unique styles for elements through selectors, properties, and values.

Problem Definition. This work addresses the design-to-code task, where the input is a visual design of a webpage, and the goal is to generate HTML+CSS code that accurately reproduces it. Let I_0 be the design image of a webpage, and C_0 be the ideal HTML+CSS code. Given I_0, an MLLM M generates code $C_g = M(I_0)$, and the rendered output I_g of C_g should closely match I_0 in both structure and appearance.

Because design files are typically unavailable, we use screenshots of existing webpages as proxies for design images I_0, and use the corresponding HTML+CSS code as ground truth C_0. This allows us to construct datasets for evaluating the model's ability to recover code from visual designs. The quality of C_g is assessed based on (1) code similarity to C_0—matching elements and structure—and (2) visual similarity between the rendered output I_g and the design image I_0.

3 Dataset Collection

This section illustrates how we develop a dataset containing representative websites for this study. We randomly sample websites from the top 500 listed on the Tranco[1] [57], and process them. First, we filter out all invalid websites (e.g., blocked, empty, or requiring human verification). Next, to make the websites self-contained and eliminate external dependencies that affect the UI code generation, we save each one into a single HTML file using the SingleFile toolkit[2], remove all external links, and replace images with placeholders. Finally, we clean the websites by removing all elements that do not impact the website's appearance (i.e., hidden elements). The dataset collection yields 348 websites containing an average of 26 unique tags, where each HTML code file has an average length of 99,431 tokens.

Table 1 provides quantitative metrics to evaluate the complexity and comprehensiveness of our dataset. **(1) Length:** We tokenize the HTML code files in each category using the GPT-2 tokenizer. The average number of tokens is 99,431, which suggests that our dataset covers a wide range of HTML code complexity. **(2) Total number of tags:** We counted the number of tags in HTML files, finding an average of 961, which provides insights into the structural complexity of the documents. **(3) DOM tree depth:** We calculated the Document Object Model (DOM) tree depth in HTML

[1] https://tranco-list.eu/. Accessed in Jan 2024
[2] https://github.com/gildas-lormeau/SingleFile

Table 1. Statistics of the dataset.

	Min	Max	Average
Length (tokens)	383	868,108	99,431
Tag Count	2	16,331	961
DOM Depth	2	40	17
Unique Tags	2	66	26
Total size			348

Table 2. Distribution of common mistakes (one element can have multiple mistakes).

Category	Percentage (%)	Count
Omission	85.34	1,450
Distortion	2.56	44
Misarrangement	12.71	216
Correct	2.35	40
Total		1,699

files. The average depth is 17, indicating a high nesting complexity of HTML tags. **(4) Number of unique tags:** We also counted the number of unique tags, with an average of 26, reflecting the diversity and content richness of the HTML files.

4 Motivating study

We begin with a motivating study to examine MLLMs' ability to generate UI code from screenshots. Taking GPT-4o as an example, we showcase three common mistakes MLLMs make during the generation and further discuss how to mitigate these errors.

4.1 Study Preparation

To explore MLLM's performance in the design-to-code task, we compare the original website's appearance with the version generated by MLLM (i.e., GPT-4o in this section). This involves an image-based comparison to check if the *visual elements* from the original are accurately reproduced in the MLLM-generated website, after rendering.

4.1.1 Dataset sampling. The pilot study samples 12 top-ranked websites from the dataset described in Section 3, such as Amazon, Apple, and Facebook. After sampling, we prompt GPT-4o to generate website code from the screenshot using the direct prompt described in Section 6.1.4.

4.1.2 Visual Element Locating (for original websites). This step identifies the bounding boxes for each element (tag) in the code within the webpage screenshot, referred to as visual elements. Using Selenium WebDriver[3], we extract the bounding box coordinates for all elements and eliminate any bounding boxes that intersect. This collection of bounding boxes enables us to locate all HTML elements in a website's screenshot. Overall, these webpages contain a total of 1,699 HTML elements, with individual webpages featuring between 4 and 434 elements, averaging 142 elements per site.

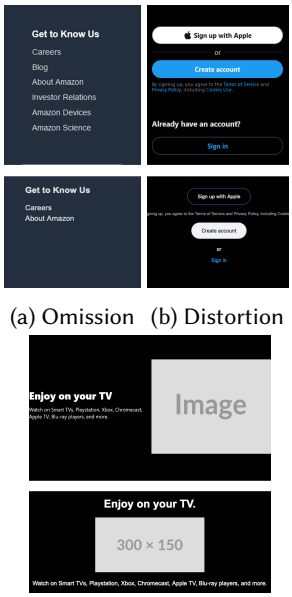
4.2 What failures does MLLM produce when converting design to code?

We recruit two graduate students in our university as annotators to assess the accuracy of visual elements in MLLM-generated webpages compared to the original ones. The annotators are provided with pairs of screenshots, with the original on the left and the MLLM-generated version on the right. A red bounding box highlights the visual element in the original screenshot. Annotators are asked with two tasks: (1) Draw the bounding box around the corresponding element in the MLLM-generated screenshot. (2) Describe any differences between the generated visual element and the original in short phrases.

After two annotators independently complete these tasks, we take the union of the two annotators' descriptions as the final annotation for each element and manually analyze the descriptions.

Result Analysis. After inspection, we discover that all the differences between the generated element and the original element can be characterized into three categories, that is, element

[3]https://selenium-python.readthedocs.io/



(a) Omission (b) Distortion

(c) Misarrangement

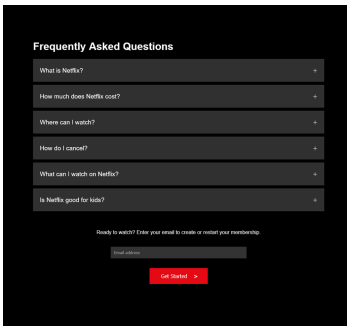
Fig. 1. Examples of error cases (bottom of each subfigure).



(a) Input. The full image is the input of (b) and the yellow dashed line area is the input of (c).



(b) Generated yellow dashed line area when input is the full screenshot.



(c) Generated yellow dashed line area when input is the area's screenshot.

Fig. 2. An example of the experiment in the motivating study.

omission, element distortion, and element misarrangement, and one element can have multiple errors at the same time. Figure 1 provides an example of each type of error, with the original website displayed above and the MLLM-generated version below in each subfigure.

(1) Element omission: Certain visual elements are missing in the generated code. As shown in Figure 1a, the text elements below "About Amazon" are missing.
(2) Element distortion: Elements are reproduced inaccurately in terms of shape, size, or color. Figure 1b illustrates a case where the "sign up with Apple" button is incorrectly colored.
(3) Element misarrangement: Elements are not positioned or ordered correctly relative to their adjacent elements or the overall design layout. In Figure 1c, MLLM misplaces the text "Enjoy on your TV" in relation to the image.

The distribution of these errors is shown in Table 2, showing that element omission accounts for the majority of error cases, which is potentially due to the visual shortcomings of MLLMs [63]. In total, MLLM correctly reproduces only 40 out of 1,699 elements, highlighting the need for a practical UI code generation approach.

4.3 Can we correct mistakes by visual segments?

After noting the unsatisfactory performance in generating precise UI code, we investigate the reasons for these errors. LLMs are known to excel at simple, small-step tasks, and struggle with long-term planning for complex tasks [67]. To address this, existing studies have developed the Chain-of-Thought paradigm or LLM Agents, which decompose complex assignments into smaller, manageable sub-tasks, enabling models to perform more reasoning steps [29].

Inspired by these studies, we break down full screenshots into smaller image segments to investigate whether MLLMs perform better with simpler inputs. This approach helps us determine

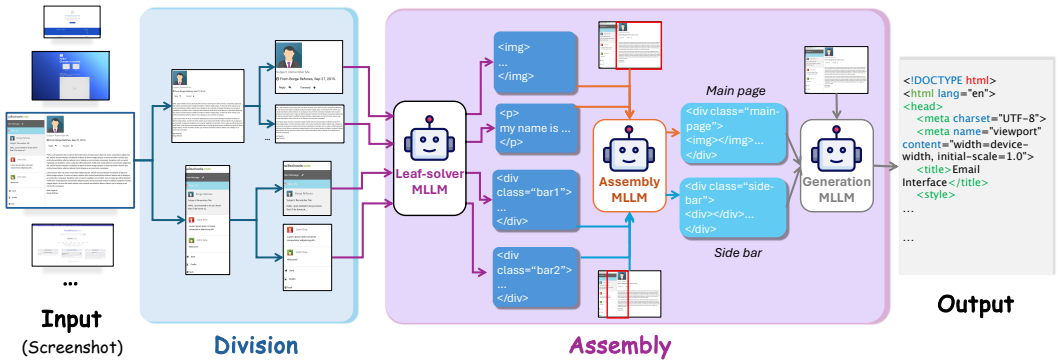


Fig. 3. The framework of DCGen.

whether the model is fundamentally incapable of generating the content or if its mistakes can be alleviated. In specific, we conduct follow-up investigations on two failure cases.

The first case focuses on omitted elements. We isolate each missing element by cropping the screenshot to include only that specific element and then request the model to generate the corresponding code. Figure 2 depicts this process, where Figure 2b is the Frequently Asked Questions (FAQ) area generated from the full screenshot, and Figure 2c is the FAQ area generated from the cropped screenshot. The results show that with these isolated inputs, the model successfully generates the previously omitted content. This demonstrates that the model is intrinsically capable of producing the required outputs but struggles with complex inputs.

The second investigation concerns distorted and misarranged elements. We compare the image similarity between the original elements and the model-generated elements from both full and cropped screenshots. We measure image similarity using the CLIP embeddings [58], as outlined in [62]. The findings reveal an overall improvement in similarity scores from 73.7% to 76.0% by providing the cropped screenshot, confirming that smaller and more focused images can enhance the quality of the generated code.

Insights: While existing MLLMs struggle with generating accurate UI code, we observe that breaking down full screenshots into smaller visual segments improves performance. This decomposition allows models to conduct more reasoning steps, each focused on a manageable sub-generation task.

5 Methodology

In this section, we present DCGen, an end-to-end framework for automated UI code generation from design images. Motivated by the traditional "*divide and conquer*" algorithmic technique, DCGen decomposes a complicated screenshot into smaller, doable visual segments, solves each part individually, and then combines the solutions for the original problem.

DCGen accepts website screenshots and outputs a file of UI Code. Figure 3 shows the overall workflow of DCGen, consisting of two stages: *division* and *assembly*. In the division stage, DCGen searches for horizontal and vertical separation lines in an interleaved fashion and splits the image accordingly, breaking the image into smaller pieces until no more separation lines are found or a user-defined maximum depth is reached. At this point, DCGen passes each image segment to

MLLMs for code generation. Afterward, in the assembly stage, code from smaller, child segments is progressively integrated to build up their parent segments. This recursive assembly continues until the full structure of the website is restored.

5.1 Division

Given a full screenshot, the division stage outputs a hierarchical structure of image separations, with each leaf node representing a fully subdivisioned image (i.e., the smallest segment). The screenshot division faces two challenges. First, the division algorithm must carefully examine visual elements to ensure that no single element is split into multiple segments. Second, the image segments should be stored in order, so that they can be reconstructed for the full webpage later.

The following section delineates a novel division algorithm, which comprises two main components: separation line detection and screenshot subdivision. The separation line detector first identifies all explicit or implicit separation lines within screenshots (either vertical or horizontal). Then, the screenshot subdivision component leverages these lines to segment the image into smaller and hierarchical segments. The two components work alternately, dividing the image horizontally, then vertically for the subimages.

5.1.1 Separation Line Detection. DCGen starts with identifying all separation lines (either vertical or horizontal) in a full screenshot. These separation lines must divide the screenshot into rectangular and meaningful pieces, where each of the pieces contains elements that are as complete as possible; otherwise, it would be more difficult to combine code for fractured pieces later. Traditional image segmentation techniques concentrate on distinguishing irregularly shaped objects from the background [48], becoming unsuitable for dividing webpages.

We observe that websites contain explicit (visible lines separating content) or implicit (blank spaces or borders between content elements) separation lines to organize different sections and contents. Figure 4 depicts examples of both types. Figure 4a employs explicit lines to separate the grey area from the purple area, while Figure 4b adapts implicit lines to arrange text sections.

Motivated by two types of separation lines, our algorithm (shown in Algorithm 1) begins by converting the screenshot to grayscale, and then scans the screenshot row by row to identify explicit lines and implicit lines, respectively. The separation lines are signified by sudden shifts in pixel values.

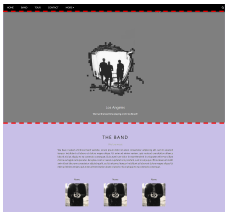
For detecting implicit separation lines, our approach incorporates a sliding window technique (lines 3-5), analyzing groups of consecutive rows to detect zones of minimal brightness variation, indicating potential blank spaces (lines 6-7). The algorithm compares each window against the rows directly above and below (lines 8-11). A separation line is identified if (1) the variance within the window is lower than a set threshold, suggesting a blank area, and (2) the brightness difference between this window and the rows directly above and below exceeds another threshold over more than *portion_thr* of the row's length (line 9, 11). This dual-threshold method facilitates the identification of subtle borders and blanks, recognizing both the low-content areas and the significant transitions at their edges. The selection of thresholds and parameters are detailed in Section 7.1.

For detecting explicit separation lines, the method is equivalent to finding implicit separation lines by setting the sliding window size to one. In particular, the algorithm scrutinizes each row for uniformity in pixel brightness. A row with variability below a predefined threshold is flagged as a potential line. Confirmation of a separation line occurs if the average difference in brightness between this row and its predecessor or successor surpasses a set threshold over at least *portion_thr* of the row.

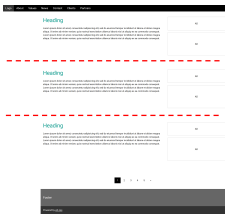
Algorithm 1 Separation Line Detection Algorithm

```
Require: img, var_thr, diff_thr, portion_thr, window_size
lines ← []
for i ← window_size + 1 to len(img) − 1 do
    upper ← img[i - window_size - 1]
    window ← img[i - window_size : i]
    lower ← img[i]
    var ← variance(window)
    is_blank ← (var < var_thr)
    diff_top ← abs_diff(upper, window[0])
    is_border_top ← portion(diff_top > diff_thr) > portion_thr
    diff_bottom ← mean_abs_diff(lower, window[-1])
    is_border_bottom ← portion(diff_bottom > diff_thr) > portion_thr
    if is_blank and (is_border_top or is_border_bottom) then
        if is_border_bottom then
            pos ← i
        else
            pos ← i - window_size
        end if
        lines.append(pos)
    end if
end for

return sorted(lines)
```



(a) Explicit.



(b) Implicit.

Fig. 4. Separation lines (red dot lines).

5.1.2 Screenshot Subdivision. We divide the screenshot into smaller segments by *alternately and recursively* applying horizontal and vertical divisions to mirror real-world development practices [19]. This process is applied recursively to each resulting segment until no further separation lines can be identified or until a user-defined maximum recursion depth is reached.

We employ a tree structure to store the hierarchical screenshot divisions. If the image I is divided into segments I_a and I_b, then I is considered the parent of segments I_a and I_b, with I_a and I_b as its children. Consequently, the number of recursions corresponds to the number of layers in the tree. Each leaf node represents a fully subdivisioned screenshot, which is the smallest segment.

5.2 Assembly

In the assembly process, we integrate the solutions of image subdivisions to generate complete UI code for the original screenshot. The assembly process is essentially the reverse of the division procedure, where code from smaller child segments is progressively integrated to build up their parent segments until the full structure of the website is restored.

We propose two different implementations of the assembly process, agent-based and rule-based. Agent-based assembly is shown in Figure 3. DCGen first provides the MLLM (i.e., Leaf-solver MLLM) with leaf image segments, then asks it to generate the code of the segments. Then, DCGen provides the MLLM (i.e., Assembly MLLM) a parent segment and all the code generated from its children's image segments. In the last step (merging for root node), DCGen calls the Generation MLLM to produce the complete UI code for full screen based on the code from its children segments and the entire screenshot. We adapt the basic instructions based on the previous study by Si et al. [62]. All the prompt details for the leaf-solver MLLM, assembly MLLM, and generation MLLM are made available in our artifact. Algorithm 2 shows the pseudocode of the complete DCGen algorithm.

For rule-based assembly, instead of using multiple MLLMs to assemble partial code output at different depths, we replace the assembly step with a rule-based program that uses CSS grid for layout positioning[4]. As usual, we first divide the input image into multiple segments. The position

[4]https://www.w3schools.com/css/css_grid.asp

Algorithm 2 DCGen-Agent

Input: img, bot, $depth$, max_depth, $leaf_prompt$, $node_prompt$
if $depth \geq max_depth$ **then**
  return $bot.generate_code(leaf_prompt, img)$
end if
$img_cuts \leftarrow segment_image(img)$
if no cuts found **then**
  return $bot.generate_code(leaf_prompt, img)$
end if
$code_parts \leftarrow [\,]$
for each cut in img_cuts **do**
  $code_parts \leftarrow$ run in new thread: $dcgen(cut, bot, depth + 1, max_depth, leaf_prompt, node_prompt)$
end for

return $bot.generate_code(node_prompt + code_parts, img)$

Algorithm 3 DCGen-Rule

Input: img, bot
$img_segs, seg_positions \leftarrow segment_image(img)$
$code_segs \leftarrow generate_code_parallel(img_segs, bot)$
$HTML_template \leftarrow CSS_grid(seg_positions, codes_segs)$
$final_code \leftarrow generate_code(HTML_template, img, bot)$

return $final_code$

Fig. 5. Statistics of the test data

	Min	Max	Average
Length (tokens)	383	123,578	28,430
Tag Count	2	396	188
DOM Depth	2	28	12
Unique Tags	2	57	19
Total size			111

of each segment within the overall layout is recorded. Then, MLLMs are used only at the leaf level to independently generate HTML and CSS code for each segment. Consequently, instead of using LLMs to assemble the segments into a full webpage, we directly place each generated segment into a CSS grid layout at the recorded positions. The assembled template, now containing all the fine-grained code blocks in the correct positions, is passed to a generator LLM for final adjustments, ensuring cohesion, flexibility, and accurate visual details. Algorithm 3 shows the pseudocode of this implementation.

6 Experiment

We evaluate the effectiveness of DCGen for the design-to-code performance by answering the following research questions.

- **RQ1:** How do different parameters influence the performance of the separation algorithm?
- **RQ2:** How does DCGen compare to various methods in design-to-code?
- **RQ3:** How robust is DCGen for complex designs?
- **RQ4:** How generalizable is DCGen for other state-of-the-art (SOTA) MLLMs?
- **RQ5:** What is human's feedback on DCGen in real-world application development workflows?

6.1 Experiment Setup

6.1.1 Backbone Models. To assess the performance of DCGen, we employ it to evaluate four SOTA MLLMs: GPT-4o [55], Gemini-1.5-flash and Geimini-2-flash [32], Claude-3-Sonnet [7]. Access to all models is facilitated through their respective official APIs. The specific model numbers are 20240513 for GPT-4o, 20240229 for Claude-3-Sonnet, Gemini-1.5-flash-latest accessed during May 2024, and Gemini-2.0-flash accessed Feb 2025. Due to a limitation on computational resources and budget, we randomly sampled 111 websites from our dataset to form the testing set. Table 5 shows the statistics of the testing dataset. The data is rich in HTML tags and diverse in overall complexity.

6.1.2 Parameter Setting. For MLLM model configurations, we set the temperature to 0 and the *max_tokens* parameter to 8192 for each model, while other parameters are maintained at their default settings as specified in the corresponding API documentation [8, 33, 56]. For parameters in the image segmentation algorithm detailed in Section 5.1, we set *var_thr=50*, *diff_thresh=45*, *diff_portion=0.9*, *window_size=50* and set the division depth to 2.

6.1.3 Efficiency. DCGen leverages parallelism to promote efficiency, where all queries at the same depth are processed simultaneously, as shown in Algorithm 2. Given an MLLM response time of t per query, the total response time for DCGen is $(depth + 1) * t = 3t$ for $depth = 2$. As for cost efficiency, on each depth where MLLM produces partial code for an image segmentation, the sum of the output code on each depth is roughly upper bound by the length of the final generated code L. Thus, the total token usage (and cost) for agent-based assembly is upper bounded by $(depth + 1) * L = 3L$ for $depth = 2$. In short, DCGen costs three times that of direct querying. However, with the rule-based assembly, DCGen is able to eliminate intermediate MLLM queries by generating code at the leaf and root levels. In such a way, the optimal cost can be lowered to $2L$, which stays the same as self-refinement prompting.

6.1.4 Baselines. We first compare our method with the Pix2Code method [11] that trained deep-learning model to convert screenshot to code. Then we compare various prompting strategies including Chain-of-Thought (CoT) prompting [66] that generates a chain of thought for each question and then generates the corresponding code. For CoT, we use the "let's think step by step" instruction from Chae et. al [12]. Self-refine prompting [18] let the model refine its own generated code via multi-turn conversation. We adopt the self-refine prompting and direct promoting method from Si et al. [62]. The details of prompting strategies used in our experiment as baselines are all available in our artifact.

6.2 Automatic Metrics

Our experiments use screenshots of existing webpages as proxies for design images I_0, and use their corresponding HTML+CSS code as ground truth C_0. We evaluate the performance of DCGen in both high-level similarity and fine-grained element matching.

6.2.1 High-Level Similarity. We utilize two high-level similarities in this paper including **code similarity** and **image similarity**, respectively.

For code similarity, we adopt the Normalized Levenshtein Distance (edit distance) [59] to evaluate the closeness of generated code C_g to the original code C_0. Levenshtein Distance is a robust method that is widely used to measure similarity between two sequences [35]. The score is calculated as $S = 1 - distance/(l_1 + l_2)$ [59], which provides a direct measurement of similarity between the generated and ground truth HTML, with a higher value indicating higher similarity.

For image similarity, we measure image similarity between I_0 and I_g via CLIP Score [62]. CLIP score relies on the similarity of their CLIP [58] embeddings, denoted as CLIP(I_0, I_g). Specifically, we extract features using CLIP-ViT-B/32.

6.2.2 Fine-Grained Measurements. High-level metrics capture only the overall similarity of images and code, lacking the detail necessary to analyze specific model performances. To address this limitation, we adopt a suite of element-matching metrics for evaluating generated webpages in terms of text content, position, and color introduced by Si et al. [62]. Given reference and generated webpage screenshots I_0 and I_g, the algorithm detects visual element blocks and matches them using the Jonker-Volgenant algorithm. It then evaluates the similarity of the matching blocks across several aspects: (1) Block-match measures the ratio of matched block sizes to all block sizes, penalizing missing or hallucinated blocks; (2) Text similarity calculates character overlap using

the Sørensen-Dice coefficient; (3) Color similarity employs the CIEDE2000 formula to evaluate perceptual color differences; and (4) Position similarity assesses the alignment of block centers.

7 Results & Analysis

7.1 RQ1. How do parameters influence the performance of the separation algorithm?

In this section, we examine how five hyper-parameters, i.e., *diff_thresh*, *var_thresh*, *diff_portion*, *window_size*, and *depth*, affect the performance of the separation algorithm. To evaluate the precision of the separation algorithm, we extract bounding boxes formed by the separation lines and use the standard Intersection over Union (IoU) metric [60] to quantifies the overlap between predicted and ground-truth bounding boxes. The IoU between two bounding boxes b_1, b_2 is calculated as $IoU = \frac{\text{Intersection}(b_1,b_2)}{\text{Union}(b1,b2)}$, with higher value indicating more overlapping or higher precision.

For each parameter configuration, we assess the algorithm's performance using our entire test set. Specifically, given an input image, we recursively apply our separation algorithm to generate separation lines. The rectangular areas defined by these lines serve as predicted bounding boxes. We then extract ground-truth bounding boxes from the corresponding HTML code using JavaScript. For each predicted bounding box, we compute its maximum IoU with all ground-truth bounding boxes. The average IoU across the test set provides a measure of the algorithm's precision. However, it is important to note that IoU serves only as a reference metric. The goal of the separation algorithm is not to perfectly match each HTML element but to generate reasonable divisions that help MLLMs focus on finer details.

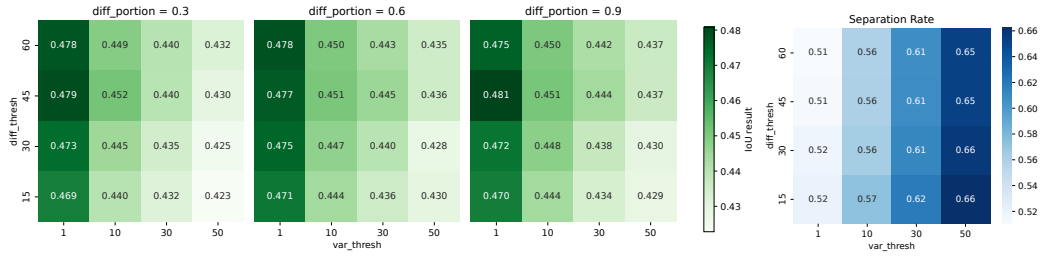
Beyond IoU, we also evaluate the extent of image division. Some parameter settings may yield highly accurate but overly coarse segmentations (e.g., producing only a few bounding boxes), while others may result in more granular yet less precise divisions. To quantify this, we introduce the Separation Rate = $1 - \frac{b_{max}}{S}$, where b_{max} represents the size of the largest undivided leaf bounding box, and S is the total image size. A higher separation rate indicates finer image segmentation.

7.1.1 Effect of diff_thresh, var_thresh, and diff_portion. To analyze the impact of *diff_thresh*, *var_thresh*, and *diff_portion*, we fixed *depth*=2 and *window_size*=50, then selected three *diff_portion* values (0.3, 0.6, and 0.9) spanning its range (0,1). For each *diff_portion*, we plotted the IoU scores across different *diff_thresh* and *var_thresh* settings.

Figure 6a shows that *diff_portion* and *diff_thresh* have minimal impact ($\leq$ 1%) on algorithm precision, whereas a lower *var_thresh* improves accuracy. Experiments indicate that IoU variations across parameter settings remain within approximately 5%, demonstrating the separation algorithm's robustness. These fluctuations arise because lower *var_thresh* reduces the likelihood of misclassifying areas as "blank", thereby limiting unnecessary separation proposals, which results in a more conservative approach, yielding fewer but more accurate separation lines. Meanwhile, as illustrated in Figure 6b, increasing *var_thresh* significantly enhances the separation rate, producing finer-grained segmentations.

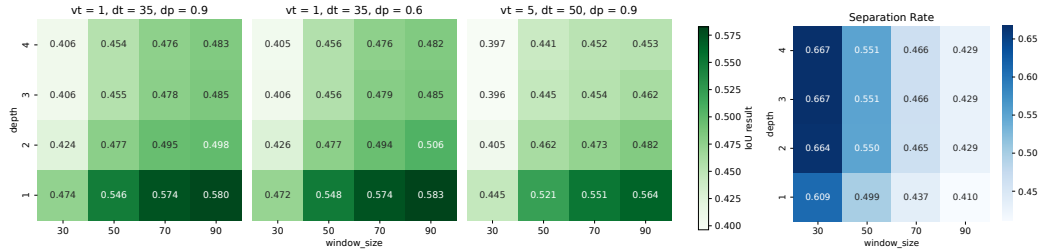
7.1.2 Influence of Window Size and Depth. To analyze the effect of sliding window size and separation depth on the separation algorithm, we selected three sets of parameters from Section 7.1.1 and plotted the average IoU across different window sizes and depths. The results, shown in Figure 7a, indicate that a smaller depth and a larger window size generally lead to more accurate separation.

On the other hand, we observe a similar trade-off between precision and separation rate. Figure 7b shows that a larger depth and smaller window size lead to a finer separation rate. However, while an increase from 1 to 2 enhances the separation rate, further increases in depth barely influence the rate.



(a) (b)

Fig. 6. (a) Effect of *diff_portion*, *diff_thresh*, and *var_thresh* on the separation algorithm's IoU score. (b) Effect of *diff_thresh*, *var_thresh* on separation rate when *diff_portion*=0.9. Larger *var_thresh* improves separation rate.



(a) (b)

Fig. 7. (a) IoU under different window sizes and depths across three sets of *var_thresh* (vt), *diff_thresh* (dt), and *diff_portion* (dp). (b) Separation rate under different window sizes and depths when vt=5, dt=50, and dp=0.9. A larger depth and smaller window size lead to a finer separation rate. Depth 2, 3, and 4 have a similar separation ratio.

7.1.3 Parameter Selection. We selected the parameters based on observations from Sections 7.1.1 and 7.1.2. Observing that *diff_portion* and *diff_thresh* have minimal influence on algorithm precision, we randomly pick *diff_portion*=0.9 and *diff_thresh*=45 for implementing DCGen. While smaller *var_thresh* improves separation accuracy, it fluctuates by only about 5%. On the other hand, larger *var_thresh* significantly improves the separation rate. Therefore, we pick a larger *var_thresh*=50. Then, to achieve a better separation rate and maintain a reasonable IoU, we pick *window size*=50. Finally, since an increase of depth from 1 to 2 obviously enhances the separation rate, further increases in depth barely influence the rate but decrease IoU significantly, we pick depth=2 to achieve a balance between precision and division rate. Figure 8 shows the trade-off we observed in Sections 7.1.1 and 7.1.2, and Figure 8c shows an example division of the parameters we adopted.

Answer to RQ1: Diff portion and diff thresh have minimal influence on the algorithm; Smaller *vt*, shallower depth, and larger *ws* produce fewer but larger and more accurately segmented areas; Larger *vt*, greater depth, and smaller *ws* generate smaller and finer divisions, albeit with reduced accuracy.

7.2 RQ2: How does DCGen compare to various methods?

This RQ assesses DCGen's performance in design-to-code generation by comparing it against various prompting strategies across multiple automatic metrics.

Table 3. DCGen's performance on GPT-4o (%).

Method	CLIP	Code
Direct	83.72	12.76
CoT	83.64	13.64
Self Ref.	83.96	13.65
DCGen	**85.62**	**15.34**
Pix2Code	73.35	9.70

Table 4. Fine-grained comparison of different methods (%).

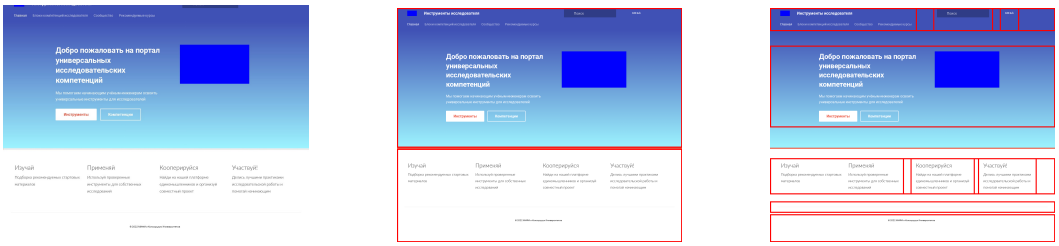
Method	Block-Match	Text	Position	Color
Direct	76.1	95.9	81.6	81.8
CoT	79.5	88.4	76.9	76.2
Self Ref.	81.4	96.1	82.7	81.9
DCGen	**81.7**	**97.8**	**83.6**	**85.4**

Table 5. DCGen + GPT-4o on Design2Code-Hard (%).

Method	CLIP	Code
Direct	81.04	14.37
CoT	79.97	14.90
Self Ref.	80.43	15.20
DCGen	**84.02**	**19.48**
Pix2Code	73.56	11.17

7.2.1 High-level Performance. Table 3 presents the high-level performance comparison of DCGen and several baseline methods across the CLIP and Code metrics. DCGen outperforms all other methods, achieving a notable 85.62% on CLIP and 15.34% on Code. Specifically, it surpasses the self-refine approach by 1.66% on CLIP and 1.69% on Code, highlighting its superior design-to-code generation capabilities. Notably, the Chain-of-Thought (CoT) method, while improving the text generation (Code: 13.64%), does not significantly outperform other methods in visual fidelity.

7.2.2 Fine-grained Metrics. Table 4 showcases the fine-grained performance comparison of the methods in terms of Block-Match, Text, Position, and Color metrics. DCGen excels in these areas, achieving the highest scores for Block-Match (81.7%), Text (97.8%), Position (83.6%), and Color (85.4%). The results suggest that DCGen provides a more accurate and complete design-to-code transformation, with particularly strong improvements in color accuracy (3.5%) and text generation (1.6%).



(a) Input image (b) IoU=0.58, Sep Rate=0.45 (c) IoU=0.43, Sep rate=0.61

Fig. 8. Comparison of different parameter settings for the separation algorithm. (b) ws=90, vt=1, dt=45, dp=0.9, depth=1. A larger *ws*, smaller *vt* and *depth* result in fewer but more accurate segmentation areas (c) ws=50, vt=50, dt=45, dp=0.9, depth=2. A smaller *ws*, larger *vt* and *depth* lead to finer but less precise segmentation (these are the parameters we used in the final experiment).

Answer to RQ2: DCGen consistently outperforms other design-to-code techniques on both high-level and fine-grained metrics, due to the focused image segments.

7.3 RQ3: How robust is DCGen for complex designs?

To evaluate the limits of DCGen, we test it on the Design-to-Code-Hard dataset, which consists of 80 highly complex webpages from GitHub Pages. This dataset presents significant challenges for state-of-the-art MLLMs in converting visual designs into code [62]. Despite the increased difficulty, DCGen achieves the best performance in both code and visual similarity, as shown in Table 5.

Table 6. Performance comparison across different input image sizes (in million pixels). All metrics are scaled by 100. Prompt$_{max}$ denotes the maximum score among self-refine, CoT, and direct prompting. "DCGen Advantage" indicates performance improvement of DCGen over Prompt$_{max}$, with positive values highlighted in green and advantages greater than 5 underlined.

Image size (M)	(0, 5]	(5, 10]	(10, 15]	(15, 20]	(20, 25]
Sep Rate	58.8	70.8	75.0	84.1	51.4
IoU	43.6	41.5	36.5	21.2	45.1
CLIP Prompt$_{max}$	84.7	83.4	76.1	72.9	75.2
CLIP DCGen	86.3	83.9	80.2	81.3	82.7
DCGen Advantage	1.6	0.5	4.1	8.4	7.5
Code Prompt$_{max}$	15.9	11.3	5.5	11.5	4.5
Code DCGen	18.2	15.7	5.9	27.1	6.5
DCGen Advantage	2.3	4.4	0.4	15.6	2.0

To further understand DCGen's role in improving results on complex webpages, we quantify website complexity using multiple metrics: image size, DOM depth, and the number of HTML tags, and focus on visual metrics (CLIP score) and code-generation metrics. We further include the Design-to-Code-Hard dataset, comprising 80 particularly challenging webpages, alongside our original dataset to provide a comprehensive assessment. The results measured by DOM depth and the number of HTML tags are presented on our website, which shows a moderate improvement of around 2% across different complexities. The results measured by image size are summarized in Table 6:

(1) The separation rate remains high (≥50%) across various complexity levels.
(2) IoU scores are relatively stable (30%-40%) across most complexity scenarios, with a notable dip in the image size interval (15, 20].
(3) When measuring complexity by image size, DCGen consistently outperforms existing prompting methods, particularly for large images (15M-25M pixels), despite the IoU dip observed.

These findings confirm that DCGen primarily benefits the conversion of large design images into code by breaking complex screenshots into smaller, more manageable visual segments. Even with imperfect segmentation, this strategy helps MLLMs better concentrate on detailed sub-tasks, significantly reducing omissions, distortions, and misarrangements. This finding is consistent with our motivating study, which revealed that decomposing large screenshots into smaller visual components improves model performance by facilitating focused reasoning steps.

Answer to RQ3: DCGen is robust to complex websites. It primarily benefits the conversion of large design images into code by breaking complex screenshots into smaller, more manageable visual segments.

7.4 RQ4: How generalizable is DCGen for other SOTA MLLMs?

In this RQ, we examine the generalizability of DCGen by employing the methodology on different MLLMs as backbones. Tables 7 and 8 display the overall performance of DCGen on Gemini-1.5-flash and Gemini-2.0-flash. Our results indicate that DCGen is highly adaptable to the Gemini-1.5-flash model, achieving notable gains of 3.9% in visual level metrics. For Gemini-2.0, DCGen enhances the visual similarity between the original and the generated websites by 0.9%, outperforming competing methods. For Claude-3-Sonnet, DCGen also enhances the visual similarity by 3.8%, outperforming

competing methods. As for code similarity, we observe that self-refine prompting can help Claude-3 preserve text content in the screenshots (e.g., paragraphs and headings), leading to a performance jump for code similarity under the self-refine prompt. Overall, the experiment demonstrates the generalizability of DCGen on various MLLMs, showing the superiority of the divide-and-conquer strategy on UI Code generation.

Answer to RQ4: DCGen is effective across different MLLMs in generating UI code from design, which demonstrates the generalizability of the proposed divide-and-conquer strategy.

Table 7. Gemini-1.5-flash.

Gemini	CLIP	Code
Direct	72.0	16.6
CoT	72.1	16.8
Self-Refine	73.7	16.1
DCGen	**77.6**	**17.8**

Table 8. Gemini-2.0-flash.

Gemini	CLIP	Code
Direct	86.7	17.2
CoT	86.4	17.5
Self-Refine	86.5	17.5
DCGen	**87.4**	**18.4**

Table 9. Claude-3-Sonnet.

Claude-3	CLIP	Code
Direct	73.8	13.5
CoT	73.2	13.3
Self-Refine	77.9	**16.5**
DCGen	**81.7**	13.5

7.5 RQ5: What is the human's feedback on DCGen in real-world application development workflows?

While automatic metrics offer a detailed breakdown of model performance, it is crucial to consider human feedback to validate the effectiveness of DCGen. To this end, we conduct a series of *human evaluations* to compare the performance of different methods and directly assess the practical utility of DCGen in real-world application development workflows.

7.5.1 Pairwise Method Comparison. In this study, we recruit five PhD students from our university as annotators. Following the conventional practice of evaluating instruction following LLMs [28, 75], we ask the annotators to compare pairs of generated webpages (one from the baseline, the other from the tested methods) to decide which one is more similar to the reference. We use GPT-4o Direct Prompting as the baseline and collect the other three methods' Win/Tie/Lose rates against this baseline. Each pair will count as Win (Lose) only when Win (Lose) receives the majority vote (≥ 3). All other cases are considered Tie.

Results. We conducted pairwise comparisons between DCGen and baseline methods, with the results shown in Figure 9. DCGen outperformed direct prompting in 62% of cases and underperformed in just 8%, showing its effectiveness in generating UI code. Meanwhile, we find that DCGen is substantially better than other baselines, while both self-revision and CoT prompting show marginal improvements over direct prompting (winning in 38-40% cases and losing in 16-18% cases). These findings suggest that automatic metrics align with human perception, demonstrating that DCGen can produce more visually similar target webpages than competing methods under human evaluation.

7.5.2 Usefulness Evaluation. While the pairwise evaluation offers a visual comparison among methods, we further assess the practicality of DCGen in accelerating the development workflow. In specific, we implement DCGen into a user-friendly demo tool (detailed in Section 8) and recruit four PhD students and research staff from our institution to participate in our study, following the methodology from Chen et al. [13]. Each participant is tasked with converting two website UI images into corresponding UI code. The two images are selected to represent different levels

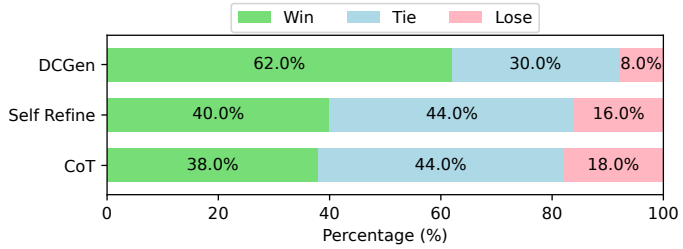
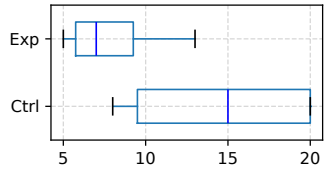
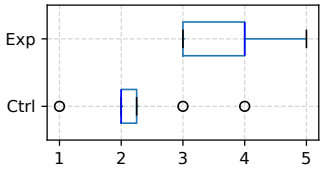


Fig. 9. Human pairwise preference evaluation results with GPT-4o Direct Prompting as the baseline. A higher win rate and lower lose rate suggest better quality as judged by human annotators.



(a) Time distribution (minutes).



(b) Similarity distribution.

Measures	Ctrl	Exp
Time (min)	14.5	**8**
Similarity	2.25	**3.75**

(c) Average result.

Fig. 10. The comparison of the experiment (Exp) and control group (Ctrl) on time and similarity.

of complexity: one simple and one more complex. Participants are randomly assigned to one of two groups: the experimental group (P1, P2), which uses DCGen to implement the UI code, and the control group (P3, P4), which is allowed to use any preferred tool (e.g., GPT-4o). A pre-study background survey confirms that all participants have similar development experience, being familiar with HTML and CSS and having previously developed one to two web applications for their work. Each participant has up to 20 minutes to complete each design.

We record the time each participant takes to implement the UI designs and capture screen recordings of their development processes, which are available in our supplementary materials. After the experiment, two research staff, who are not involved in the study, evaluate the similarity of the implemented GUIs to the original UI images using a five-point Likert scale [39] (1: not similar at all, 5: almost identical). The judges are blinded to which group produces each GUI.

Results. Figure 10 highlights UI implementation results from the experiment group (w/ DCGen) and control groups (w/o DCGen). First, the box plot in Figure 10a shows that the experiment group (Exp) completes the design-to-code task twice as fast as the group without DCGen. The control (Ctrl) group also shows greater variability, with some participants taking up to 20 minutes. The average time of the control group is underestimated because one participant failed to complete a UI image within 20 minutes. In contrast, all participants in the experiment group finish all the tasks within 15 minutes. Second, as shown in Figure 10b, the experiment group consistently achieves higher similarity scores compared to the control group. The Pearson Correlation Coefficient between the similarity scores given by the two judges in Figure10b is 0.71, suggesting a strong agreement between the judges. Finally, Figure 10c summarizes these results, confirming that the experiment group using DCGen has lower completion times and higher similarity scores on average, indicating our tool can help the experimental group implement UI images faster and more accurately.

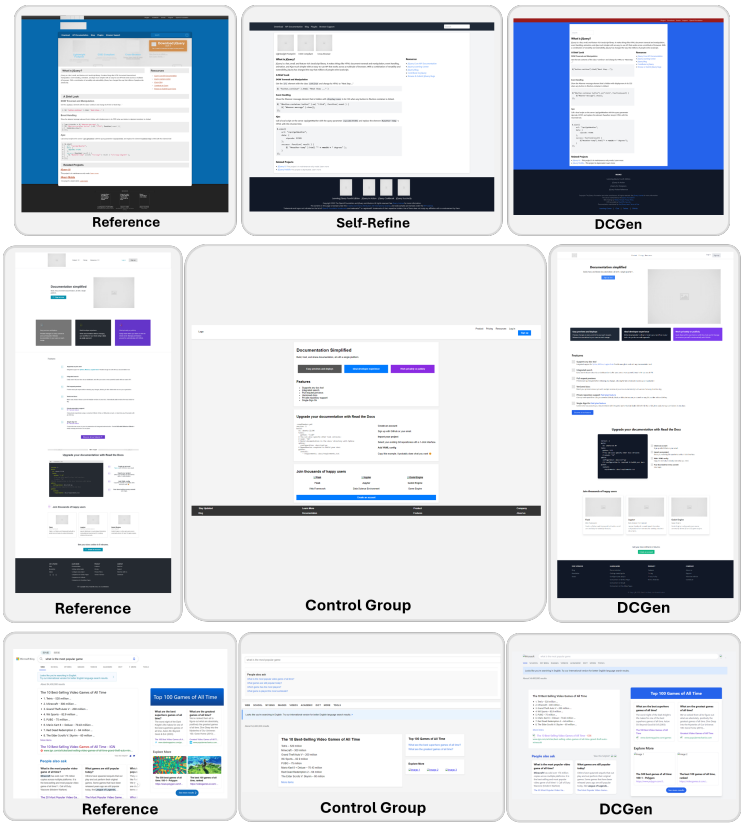


Fig. 11. Success cases. The first row shows an example from RQ2, and the rest shows examples from the usefulness study (RQ5).

> **Answer to RQ5:** Human evaluations demonstrate that DCGen can produce more visually similar target webpages than competing methods, and the DCGen tool can significantly help developers implement webpages faster and more similar to the UI designs.

8 Discussion

Based on the users' experience of interacting with DCGen, we discuss how it could contribute to transferring UI designs to code for developers, as well as identify potential limitations of DCGen for future improvements.

Success stories. The first row of Figure 11 illustrates a success case from Section 7 RQ2, which shows that compared to the self-refine method, DCGen restores the color and layout of the webpage, and removes nonexistent pictures from the webpage thus reducing distortion. The rest of Figure 11 provides examples from the usefulness study (RQ5). DCGen demonstrates its effectiveness by significantly reducing element omissions and distortions, leading to faster development and improved webpage quality.

In the usefulness study (RQ5), we observe that both developers in the control group follow a similar iterative refinement pattern to generate UI code, even though they are unaware of DCGen's methodology. Specifically, they start by prompting the model to produce an initial webpage, then iteratively refine the unsatisfactory parts by providing more detailed prompts with a screenshot of

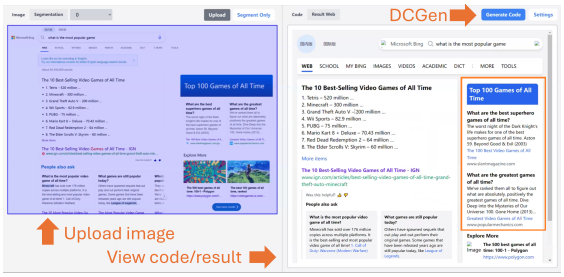


Fig. 12. Generate code for the entire webpage.

the specific areas. Such manual narrowing of task scope aligns with DCGen's automated divide-and-conquer approach.

Tool features. Figure 12 shows a screenshot of DCGen's user interface as a development tool, with the design image on the left and the code and rendered webpage on the right. DCGen enables developers to generate an entire webpage, inspect the code for individual image segments, and regenerate code for any specific segment. By automating the divide-and-conquer process, DCGen eliminates the need for detailed, repetitive prompts, speeding up development and enabling the creation of higher-quality webpages in less time compared to the control group.

Failure analysis. While DCGen effectively reduces element omission and distortion, its improvement in addressing element misarrangement is limited. A failure case is illustrated in Figure 12, within the orange rectangular area. In the original image, the paragraphs are horizontally arranged, but the generated paragraphs are vertically aligned. Upon further investigation, we found that this error occurs during the assembly process, as the leaf solver LLM correctly generates horizontally arranged paragraphs for this area, while the assembly LLM mistakenly changes the paragraphs to a vertical layout. We attribute the issue to the inherent visual limitations of MLLMs, as identified by Tong et al. [63], which suggest that MLLMs still face challenges in recognizing certain simple visual patterns, such as positional arrangements.

However, since DCGen allows users to review and regenerate code for any specific image segment, the erroneous code segment can be easily replaced with the correct version in the final webpage. Thus, this failure case has minimal impact on DCGen's practical effectiveness.

9 Threat to Validity

The selection of backbone models. This study employs four popular instruction-following and multimodal LLMs to demonstrate the effectiveness of DCGen. While other LLMs might be adapted for our methodology, we found that smaller parameter models fail to perform complex tasks such as the assembly process. Our future work involves extending our implementation to other emerging models, showcasing DCGen's generalization abilities.

Robustness of set thresholds and chosen separation strategy. Assumptions regarding the segmentation strategy and manually set thresholds may limit DCGen's applicability in more complex designs. To address this concern, we conduct RQ2 to evaluate DCGen's robustness to complex designs. The results show that DCGen delivers superior performance across various website complexities, indicating its robustness in handling both simple and intricate webpages.

10 Related Work

10.1 Image-to-code Generation

The most informative AI techniques used for generating code from images can be classified into three broad categories: (i) Convolutional Neural Networks (CNNs) [10, 11, 13, 17, 21, 49, 70], (ii)

Computer Vision (CV) and Optical Character Recognition (OCR) [51, 52], and (iii) other deep learning models [5, 68]. The work [11] utilizes CNNs and LSTM to extract features from GUI images to generate a domain-specific language (DSL). However, DSLs are not widely used in practical development, which inconveniences the industry's UI developers [70]. An alternative approach [13] using CNNs to extract visual features from UI images is more widely applicable than generating fixed DSL code. Based on CNNs, Andre et al. [21] pioneered using Class Activation Mapping (CAM), further improving the performance and interpretability of the image-to-code generation task. More recently, Interaction2Code [69] explored interactive web code generation from design images, and MRWeb [65] investigated the generation of multi-page resource-aware web code from UI designs, which extends page-level image-to-code generation to project-level.

10.2 Multimodal Language Models

Multimodal Language Models (MLLM) refer to LLM-based models capable of receiving, reasoning, and outputting multimodal information [72]. In recent years, the trend of using LLMs as decoders in vision-language tasks has gained significant traction. Pioneering studies such as VisualGPT [15] and Frozen [64] have demonstrated the advantages of employing a pre-trained language model as a decoder. Following this, Flamingo [6] was developed to align a pre-trained vision encoder and language model using gated cross-attention. Furthermore, BLIP-2 [42] introduced the use of Flan-T5 [20] with a Q-Former to efficiently align visual features with the language model. PaLM-E [26], featuring 562 billion parameters, was developed to integrate real-world continuous sensor modalities into an LLM. More recently, GPT-4o [55] and Gemini-2.5 [32] demonstrate enhanced visual understanding and reasoning abilities following pre-training on a vast collection of aligned image-text data.

11 Conclusion

In this paper, we first present a motivating study that identifies prevalent failures in MLLMs during the design-to-code generation process. Drawing on these insights, we design and implement DCGen, a novel divide-and-conquer-based framework for effectively generating UI code from web screenshots. We evaluate DCGen across three SOTA MLLMs and demonstrate that it consistently outperforms various prompting strategies. Furthermore, DCGen shows robustness against variations in website complexity and is generalizable across different MLLMs, demonstrating its superiority in generating UI code.

Data Availability

All the code, data, and user-friendly tool for DCGen are available at https://github.com/WebPAI/DCGen for replication and future research.

Acknowledgment

This research was supported by the Singapore Ministry of Education (MOE) Academic Research Fund (AcRF) Tier 1 grant and Research Grants Council of the Hong Kong Special Administrative Region, China (No. SRFS2425-4S03 of the Senior Research Fellow Scheme).

References

[1] 2024. 17+ Surprising WordPress Statistics You Should Not Miss [2024]. *WPDeveloper* (2024). https://wpdeveloper.com/wordpress-statistics-2024 Accessed: 2024-05-30.

[2] 2024. Differences Between Mobile App and Web Development. *KnowledgeHut* (2024). https://www.knowledgehut.com/blog/programming/differences-mobile-app-web-development

[3] 2024. How Many Websites Are There in 2024? (13 Latest Statistics). *TechJury* (2024). https://techjury.net/blog/how-many-websites-are-there/ Accessed: 2024-05-30.

[4] 2024. Website & Mobile App Design: Differences In 2024? *Inkbot Design* (2024). https://inkbotdesign.com/differences-website-mobile-app-design/

[5] A. A. Abdelhamid, S. R. Alotaibi, and A. Mousa. 2020. Deep learning-based prototyping of android GUI from hand-drawn mockups. *IET Software* 14, 7 (2020), 816–824.

[6] Jean-Baptiste Alayrac, Jeff Donahue, Pauline Luc, Antoine Miech, Iain Barr, Yana Hasson, Karel Lenc, Arthur Mensch, Katherine Millican, Malcolm Reynolds, et al. 2022. Flamingo: a visual language model for few-shot learning. *Advances in neural information processing systems* 35 (2022), 23716–23736.

[7] Anthropic. 2024. Claude. https://www.anthropic.com/claude Accessed: 2024-06-06.

[8] Anthropic. 2024. Vision Documentation. https://docs.anthropic.com/en/docs/vision Accessed: 2024-06-06.

[9] Shushan Arakelyan, Rocktim Jyoti Das, Yi Mao, and Xiang Ren. 2023. Exploring Distributional Shifts in Large Language Models for Code Analysis. In *Conference on Empirical Methods in Natural Language Processing*. https://api.semanticscholar.org/CorpusID:257557735

[10] Batuhan Asiroglu, Büşta Rümeysa Mete, Eyyüp Yildiz, Yagiz Nalçakan, Alper Sezen, Mustafa Dağtekin, and Tolga Ensari. 2019. Automatic HTML Code Generation from Mock-Up Images Using Machine Learning Techniques. *2019 Scientific Meeting on Electrical-Electronics & Biomedical Engineering and Computer Science (EBBT)* (2019), 1–4. https://api.semanticscholar.org/CorpusID:195223316

[11] T. Beltramelli. 2018. pix2code: Generating code from a graphical user interface screenshot. In *Proceedings of the ACM SIGCHI Symposium on Engineering Interactive Computing Systems*. 1–6.

[12] Hyungjoo Chae, Yeonghyeon Kim, Seungone Kim, Kai Tzu iunn Ong, Beong woo Kwak, Moohyeon Kim, Seonghwan Kim, Taeyoon Kwon, Jiwan Chung, Youngjae Yu, and Jinyoung Yeo. 2024. Language Models as Compilers: Simulating Pseudocode Execution Improves Algorithmic Reasoning in Language Models. *ArXiv* abs/2404.02575 (2024). https://api.semanticscholar.org/CorpusID:268876471

[13] C. Chen, T. Su, G. Meng, Z. Xing, and Y. Liu. 2018. From UI design image to GUI skeleton: a neural machine translator to bootstrap mobile GUI implementation. In *Proceedings of the 40th International Conference on Software Engineering*. 665–676.

[14] Fuxiang Chen, Fateme Moradian Fard, David Lo, and Timofey Bryksin. 2022. On the Transferability of Pre-trained Language Models for Low-Resource Programming Languages. *2022 IEEE/ACM 30th International Conference on Program Comprehension (ICPC)* (2022), 401–412. https://api.semanticscholar.org/CorpusID:248266381

[15] Jun Chen, Han Guo, Kai Yi, Boyang Li, and Mohamed Elhoseiny. 2022. Visualgpt: Data-efficient adaptation of pretrained language models for image captioning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 18030–18040.

[16] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde, Jared Kaplan, Harrison Edwards, Yura Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, David W. Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William H. Guss, Alex Nichol, Igor Babuschkin, Suchir Balaji, Shantanu Jain, Andrew Carr, Jan Leike, Joshua Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew M. Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. Evaluating Large Language Models Trained on Code. *ArXiv* abs/2107.03374 (2021). https://api.semanticscholar.org/CorpusID:235755472

[17] W.-Y. Chen, P. Podstreleny, W.-H. Cheng, Y.-Y. Chen, and K.-L. Hua. 2022. Code generation from a graphical user interface via attention-based encoder–decoder model. *Multimedia Systems* 28, 1 (2022), 121–130.

[18] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. 2023. Teaching Large Language Models to Self-Debug. *ArXiv* abs/2304.05128 (2023). https://api.semanticscholar.org/CorpusID:258059885

[19] CSS Chopper. 2024. *How to Convert PSD to HTML : A Complete Guide.* https://www.csschopper.com/blog/how-to-convert-psd-to-html-a-complete-guide/ Accessed: 31-05-2024.

[20] Hyung Won Chung, Le Hou, Shayne Longpre, Barret Zoph, Yi Tay, William Fedus, Yunxuan Li, Xuezhi Wang, Mostafa Dehghani, Siddhartha Brahma, et al. 2024. Scaling instruction-finetuned language models. *Journal of Machine Learning Research* 25, 70 (2024), 1–53.

[21] A. A. J. Cizotto, R. C. T. de Souza, V. C. Mariani, and L. dos Santos Coelho. 2023. Web pages from mockup design based on convolutional neural network and class activation mapping. *Multimedia Tools and Applications* (2023), 1–27.

[22] Wenliang Dai, Junnan Li, Dongxu Li, Anthony Meng Huat Tiong, Junqi Zhao, Weisheng Wang, Boyang Albert Li, Pascale Fung, and Steven C. H. Hoi. 2023. InstructBLIP: Towards General-purpose Vision-Language Models with Instruction Tuning. *ArXiv* abs/2305.06500 (2023). https://api.semanticscholar.org/CorpusID:258615266

[23] Victor C. Dibia, Adam Fourney, Gagan Bansal, Forough Poursabzi-Sangdeh, Han Liu, and Saleema Amershi. 2022. Aligning Offline Metrics and Human Judgments of Value of AI-Pair Programmers. *ArXiv* abs/2210.16494 (2022). https://api.semanticscholar.org/CorpusID:253237523

[24] Hantian Ding, Varun Kumar, Yuchen Tian, Zijian Wang, Robert Kwiatkowski, Xiaopeng Li, Murali Krishna Ramanathan, Baishakhi Ray, Parminder Bhatia, Sudipta Sengupta, Dan Roth, and Bing Xiang. 2023. A Static Evaluation of Code Completion by Large Language Models. *ArXiv* abs/2306.03203 (2023). https://api.semanticscholar.org/CorpusID:259088657

[25] Yihong Dong, Xue Jiang, Zhi Jin, and Ge Li. 2023. Self-collaboration Code Generation via ChatGPT. *ArXiv* abs/2304.07590 (2023). https://api.semanticscholar.org/CorpusID:258179537

[26] Danny Driess, Fei Xia, Mehdi SM Sajjadi, Corey Lynch, Aakanksha Chowdhery, Brian Ichter, Ayzaan Wahid, Jonathan Tompson, Quan Vuong, Tianhe Yu, et al. 2023. PaLM-E: An Embodied Multimodal Language Model. In *International Conference on Machine Learning*. PMLR, 8469–8488.

[27] Xueying Du, Mingwei Liu, Kaixin Wang, Hanlin Wang, Junwei Liu, Yixuan Chen, Jiayi Feng, Chaofeng Sha, Xin Peng, and Yiling Lou. 2023. ClassEval: A Manually-Crafted Benchmark for Evaluating LLMs on Class-level Code Generation. *ArXiv* abs/2308.01861 (2023). https://api.semanticscholar.org/CorpusID:260439062

[28] Yann Dubois, Xuechen Li, Rohan Taori, Tianyi Zhang, Ishaan Gulrajani, Jimmy Ba, Carlos Guestrin, Percy Liang, and Tatsunori Hashimoto. 2023. AlpacaFarm: A Simulation Framework for Methods that Learn from Human Feedback. *ArXiv* abs/2305.14387 (2023). https://api.semanticscholar.org/CorpusID:258865545

[29] Sidong Feng and Chunyang Chen. 2024. Prompting Is All You Need: Automated Android Bug Replay with Large Language Models. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*. 1–13.

[30] Shuzheng Gao, Xinjie Wen, Cuiyun Gao, Wenxuan Wang, and Michael R. Lyu. 2023. Constructing Effective In-Context Demonstration for Code Intelligence Tasks: An Empirical Study. *ArXiv* abs/2304.07575 (2023). https://api.semanticscholar.org/CorpusID:263867793

[31] Henry Gilbert, Michael Sandborn, Douglas C. Schmidt, Jesse Spencer-Smith, and Jules White. 2023. Semantic Compression with Large Language Models. *2023 Tenth International Conference on Social Networks Analysis, Management and Security (SNAMS)* (2023), 1–8. https://api.semanticscholar.org/CorpusID:258309482

[32] Google. 2024. Gemini API. https://ai.google.dev/gemini-api Accessed: 2024-06-06.

[33] Google AI. 2024. API Overview. https://ai.google.dev/gemini-api/docs/api-overview Accessed: 2024-06-06.

[34] Jian Gu, Pasquale Salza, and Harald C. Gall. 2022. Assemble Foundation Models for Automatic Code Summarization. *2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)* (2022), 935–946. https://api.semanticscholar.org/CorpusID:245986582

[35] Rishin Haldar and Debajyoti Mukhopadhyay. 2011. Levenshtein Distance Technique in Dictionary Lookup Methods: An Improved Approach. *ArXiv* abs/1101.1232 (2011). https://api.semanticscholar.org/CorpusID:1049293

[36] Xinying Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John C. Grundy, and Haoyu Wang. 2023. Large Language Models for Software Engineering: A Systematic Literature Review. *ArXiv* abs/2308.10620 (2023). https://api.semanticscholar.org/CorpusID:261048648

[37] Ronald Jabangwe, Henry Edison, and Anh Nguyen-Duc. 2018. Software engineering process models for mobile app development: A systematic literature review. *J. Syst. Softw.* 145 (2018), 98–111. https://api.semanticscholar.org/CorpusID:52961621

[38] Shuyang Jiang, Yuhao Wang, and Yu Wang. 2023. SelfEvolve: A Code Evolution Framework via Large Language Models. *ArXiv* abs/2306.02907 (2023). https://api.semanticscholar.org/CorpusID:259076266

[39] Ankur Joshi, Saket Kale, Satish Chandel, and D Kumar Pal. 2015. Likert scale: Explored and explained. *British journal of applied science & technology* 7, 4 (2015), 396–403.

[40] Kati Kuusinen and Tommi Mikkonen. 2013. Designing User Experience for Mobile Apps: Long-Term Product Owner Perspective. *2013 20th Asia-Pacific Software Engineering Conference (APSEC)* 1 (2013), 535–540. https://api.semanticscholar.org/CorpusID:18632493

[41] Valéria Lelli, Arnaud Blouin, and Benoît Baudry. 2015. Classifying and Qualifying GUI Defects. *2015 IEEE 8th International Conference on Software Testing, Verification and Validation (ICST)* (2015), 1–10. https://api.semanticscholar.org/CorpusID:2288032

[42] Junnan Li, Dongxu Li, Silvio Savarese, and Steven Hoi. 2023. Blip-2: Bootstrapping language-image pre-training with frozen image encoders and large language models. In *International conference on machine learning*. PMLR, 19730–19742.

[43] Jia Li, Ge Li, Yongming Li, and Zhi Jin. 2023. Enabling Programming Thinking in Large Language Models Toward Code Generation. *ArXiv* abs/2305.06599 (2023). https://api.semanticscholar.org/CorpusID:263896057

[44] Tsz On Li, Wen yi Zong, Yibo Wang, Haoye Tian, Y. Wang, and S. C. Cheung. 2023. Nuances are the Key: Unlocking ChatGPT to Find Failure-Inducing Tests with Differential Prompting. *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)* (2023), 14–26. https://api.semanticscholar.org/CorpusID:258298446

[45] Zongjie Li, Chaozheng Wang, Zhibo Liu, Hao Wang, Shuai Wang, and Cuiyun Gao. 2022. CCTEST: Testing and Repairing Code Completion Systems. *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)* (2022), 1238–1250. https://api.semanticscholar.org/CorpusID:251623193

[46] Antonio Mastropaolo, Luca Pascarella, and Gabriele Bavota. 2022. Using Deep Learning to Generate Complete Log Statements. *2022 IEEE/ACM 44th International Conference on Software Engineering (ICSE)* (2022), 2279–2290. https://api.semanticscholar.org/CorpusID:245906103

[47] Antonio Mastropaolo, Simone Scalabrino, Nathan Cooper, David Nader-Palacio, Denys Poshyvanyk, Rocco Oliveto, and Gabriele Bavota. 2021. Studying the Usage of Text-To-Text Transfer Transformer to Support Code-Related Tasks. *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)* (2021), 336–347. https://api.semanticscholar.org/CorpusID:231786586

[48] Shervin Minaee, Yuri Boykov, Fatih Porikli, Antonio Plaza, Nasser Kehtarnavaz, and Demetri Terzopoulos. 2021. Image segmentation using deep learning: A survey. *IEEE transactions on pattern analysis and machine intelligence* 44, 7 (2021), 3523–3542.

[49] K. Moran, C. Bernal-Cárdenas, M. Curcio, R. Bonett, and D. Poshyvanyk. 2018. Machine learning-based prototyping of graphical user interfaces for mobile apps. *IEEE Transactions on Software Engineering* 46, 2 (2018), 196–221.

[50] Kevin Moran, Boyang Li, Carlos Bernal-Cárdenas, Dan Jelf, and Denys Poshyvanyk. 2018. Automated Reporting of GUI Design Violations for Mobile Apps. *2018 IEEE/ACM 40th International Conference on Software Engineering (ICSE)* (2018), 165–175. https://api.semanticscholar.org/CorpusID:3634687

[51] S. Natarajan and C. Csallner. 2018. P2A: A tool for converting pixels to animated mobile application user interfaces. In *Proceedings of the 5th International Conference on Mobile Software Engineering and Systems*. 224–235.

[52] T. A. Nguyen and C. Csallner. 2015. Reverse engineering mobile application user interfaces with remaui (t). In *2015 30th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 248–259.

[53] Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Haiquan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. 2022. CodeGen: An Open Large Language Model for Code with Multi-Turn Program Synthesis. In *International Conference on Learning Representations*. https://api.semanticscholar.org/CorpusID:252668917

[54] OpenAI. [n. d.]. GPT-4 System Card. https://openai.com/index/gpt-4v-system-card/. Accessed: 2024-05-30.

[55] OpenAI. 2024. Hello GPT-4o. https://openai.com/index/hello-gpt-4o/ Accessed: 2024-06-06.

[56] OpenAI. 2024. Vision Guide. https://platform.openai.com/docs/guides/vision Accessed: 2024-06-06.

[57] Victor Le Pochat, Tom van Goethem, Samaneh Tajalizadehkhoob, Maciej Korczyński, and Wouter Joosen. 2018. Tranco: A Research-Oriented Top Sites Ranking Hardened Against Manipulation. *Proceedings 2019 Network and Distributed System Security Symposium*. https://api.semanticscholar.org/CorpusID:56367624

[58] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. 2021. Learning Transferable Visual Models From Natural Language Supervision. In *International Conference on Machine Learning*. https://api.semanticscholar.org/CorpusID:231591445

[59] RapidFuzz. 2024. Levenshtein Distance — RapidFuzz. https://rapidfuzz.github.io/Levenshtein/levenshtein.html Accessed: 2024-02-26.

[60] Joseph Redmon, Santosh Kumar Divvala, Ross B. Girshick, and Ali Farhadi. 2015. You Only Look Once: Unified, Real-Time Object Detection. *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (2015), 779–788. https://api.semanticscholar.org/CorpusID:206594738

[61] Raj Sanghvi. 2024. Differences Between Web and Mobile App Development. *BitCot* (2024). https://www.bitcot.com/blog/differences-between-web-and-mobile-app-development/

[62] Chenglei Si, Yanzhe Zhang, Zhengyuan Yang, Ruibo Liu, and Diyi Yang. 2024. Design2Code: How Far Are We From Automating Front-End Engineering? *ArXiv* abs/2403.03163 (2024). https://api.semanticscholar.org/CorpusID:268248801

[63] Shengbang Tong, Zhuang Liu, Yuexiang Zhai, Yi Ma, Yann LeCun, and Saining Xie. 2024. Eyes Wide Shut? Exploring the Visual Shortcomings of Multimodal LLMs. *ArXiv* abs/2401.06209 (2024). https://api.semanticscholar.org/CorpusID:266976992

[64] Maria Tsimpoukelli, Jacob L Menick, Serkan Cabi, SM Eslami, Oriol Vinyals, and Felix Hill. 2021. Multimodal few-shot learning with frozen language models. *Advances in Neural Information Processing Systems* 34 (2021), 200–212.

[65] Yuxuan Wan, Yi Dong, Jingyu Xiao, Yintong Huo, Wenxuan Wang, and Michael R. Lyu. 2024. MRWeb: An Exploration of Generating Multi-Page Resource-Aware Web Code from UI Designs. *ArXiv* abs/2412.15310 (2024). https://api.semanticscholar.org/CorpusID:274965541

[66] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Ed Huai hsin Chi, F. Xia, Quoc Le, and Denny Zhou. 2022. Chain of Thought Prompting Elicits Reasoning in Large Language Models. *ArXiv* abs/2201.11903 (2022). https://api.semanticscholar.org/CorpusID:246411621

[67] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35 (2022), 24824–24837.

[68] J. Wu, X. Zhang, J. Nichols, and J. P. Bigham. 2021. Screen parsing: Towards reverse engineering of UI models from screenshots. In *The 34th Annual ACM Symposium on User Interface Software and Technology*. 470–483.

[69] Jingyu Xiao, Yuxuan Wan, Yintong Huo, Zhiyao Xu, and Michael R. Lyu. 2024. Interaction2Code: How Far Are We From Automatic Interactive Webpage Generation? *ArXiv* abs/2411.03292 (2024). https://api.semanticscholar.org/CorpusID:273821629

[70] Y. Xu, L. Bo, X. Sun, B. Li, J. Jiang, and W. Zhou. 2021. image2emmet: Automatic code generation from web user interface image. *Journal of Software: Evolution and Process* 33, 8 (2021), e2369.

[71] Zhengyuan Yang, Linjie Li, Kevin Lin, Jianfeng Wang, Chung-Ching Lin, Zicheng Liu, and Lijuan Wang. 2023. The Dawn of LMMs: Preliminary Explorations with GPT-4V(ision). *ArXiv* abs/2309.17421 (2023). https://api.semanticscholar.org/CorpusID:263310951

[72] Shukang Yin, Chaoyou Fu, Sirui Zhao, Ke Li, Xing Sun, Tong Xu, and Enhong Chen. 2023. A Survey on Multimodal Large Language Models. *ArXiv* abs/2306.13549 (2023). https://api.semanticscholar.org/CorpusID:259243718

[73] Hao Yu, Bo Shen, Dezhi Ran, Jiaxin Zhang, Qi Zhang, Yu Ma, Guangtai Liang, Ying Li, Tao Xie, and Qianxiang Wang. 2023. CoderEval: A Benchmark of Pragmatic Code Generation with Generative Pre-trained Models. In *International Conference on Software Engineering*. https://api.semanticscholar.org/CorpusID:256459413

[74] Clemens Zeidler, Christof Lutteroth, Wolfgang Stuerzlinger, and Gerald Weber. 2013. Evaluating Direct Manipulation Operations for Constraint-Based Layout. In *IFIP TC13 International Conference on Human-Computer Interaction*. https://api.semanticscholar.org/CorpusID:8243987

[75] Chunting Zhou, Pengfei Liu, Puxin Xu, Srini Iyer, Jiao Sun, Yuning Mao, Xuezhe Ma, Avia Efrat, Ping Yu, L. Yu, Susan Zhang, Gargi Ghosh, Mike Lewis, Luke Zettlemoyer, and Omer Levy. 2023. LIMA: Less Is More for Alignment. *ArXiv* abs/2305.11206 (2023). https://api.semanticscholar.org/CorpusID:258822910

Received 2024-09-13; accepted 2025-04-01